Linear Pipelined Processor

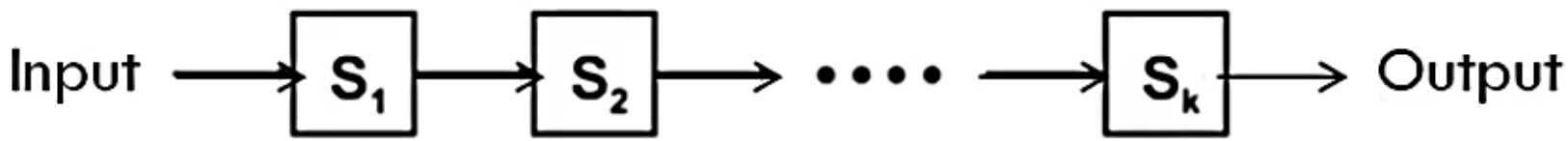
Linear Pipeline Processor is a cascade of processing stages which are linearly connected to perform a fixed function over a stream of data flowing from one end to another.

A **Linear Pipeline Processor** is **constructed** with **k processing stages.**

External **input** is supplied to the pipeline from stage **S_1.**

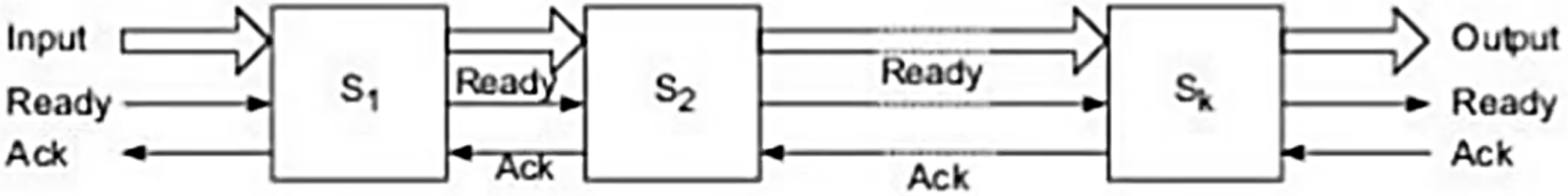
The **processed result** is passed from stage S_i to stage S_{i+1}, for all i=1, 2…., k-1.

The **final result** is produced from the last pipeline stage **S_k.**



Linear pipelined processor models are divided into two categories:

1. **Asynchronous Pipeline:** the data flow between adjacent stages are controlled by handshaking protocols.



Asynchronous pipeline model

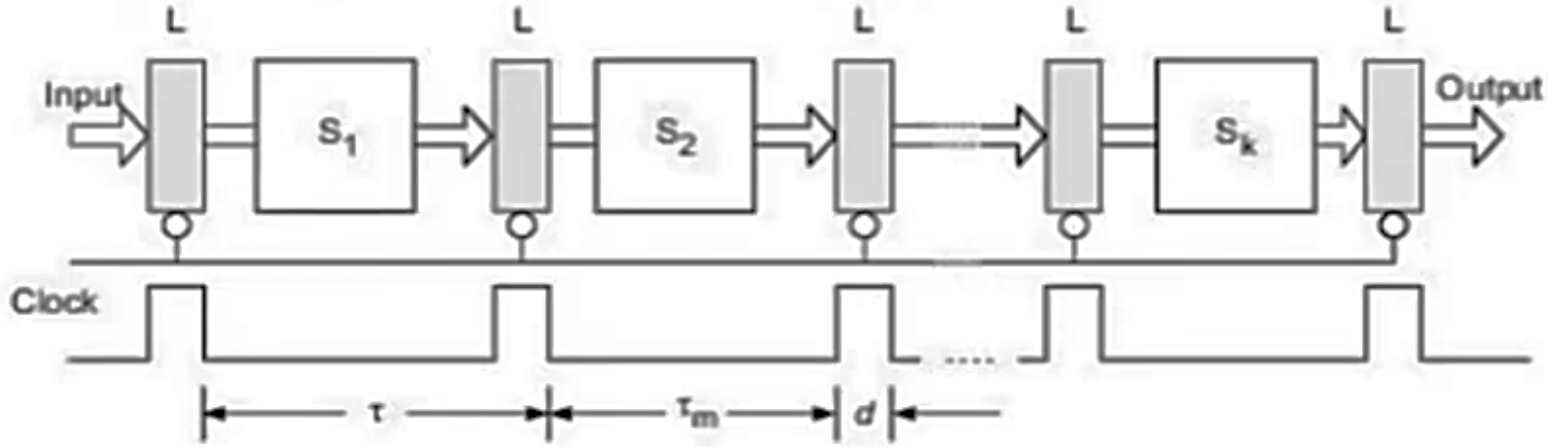
Working: When the stage S_i is ready to transmit it sends a *ready signal* to stage S_{i+1}. After S_{i+1} receives the incoming data, it returns acknowledgement signal (Ack) to S_i.

Application: Asynchronous pipelines are useful in designing communication channels in message passing multicomputer.

Asynchronous pipelines may have a variable throughput rate because different amounts of delay may be experienced in different stages.

2. Synchronous Model

In ***Synchronous pipeline***, ***Clocked latches*** are used to interface between stages.



Captions:
S_i = stage i
L = Latch
τ = Clock period
τ_m = Maximum stage delay
d = Latch delay

Synchronous pipeline model

Working:

The pipeline consists of cascade of **processing stages (Si)**. The pipeline stages are **combinational circuits** performing arithmetic or logic operations over the data stream flowing through the pipe.

The stages are separated by high speed interface **latches (L)**. The latches are fast registers for holding the intermediate results between the stages.

Upon arrival of **clock pulse** latches transfer data to next stage simultaneously.

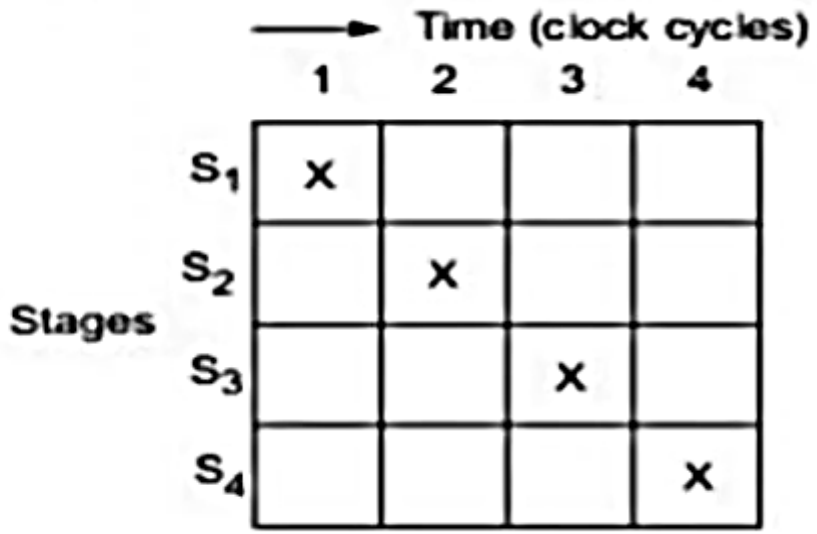
Have approximately equal delays in all stages. These delays determine the clock period and thus the speed of the pipeline

The utilization pattern of successive stages in a synchronous pipeline is specified by a ***Reservation Table***.

- In linear pipelined processors, the latches are made with master-slave FF., which can isolate input from outputs.
- Because of latches, the delay in all the stages is approximately equal which helps in determining clock period and speed of the pipeline.
- For a k-stage linear pipeline, k-clock cycles are needed for data to flow through the pipeline.

Reservation Table or space time diagram of linear pipeline

RESERVATION TABLES: Specifies the utilization pattern of successive stages in a synchronous pipeline i.e. which stage is used in which clock cycle.



For **Linear Pipeline**, the utilization follows **diagonal** stream line pattern

This **Reservation Table** is essentially a **Space-Time diagram** depicting precedence relationship in using pipeline stages

Pipelined vs. Non-pipelined Processing

Pipelined Processing:

Stages/Clock Cycles	1	2	3	4	5	6
Stage 1	T1	T2	T3			
Stage 2		T1	T2	T3		
Stage 3			T1	T2	T3	
Stage 4				T1	T2	T3

T_k = [k + (n-1)] clock cycles
= 4 + (3-1) clock cycles
= 6 clock cycles

***In Pipelining**, once the pipeline is filled up, it will output one result per clock cycle, independent of number of stages in the pipe.*

Where, k=no. of stages
n= no. of tasks

Total time to process 'n tasks' in 'k-stage pipelined processor':

$$T_k = [\, k + (n-1)\,] \text{ clock cycles}$$

*k cycles for the **first task**, and*
*n-1 cycles for the **remaining n-1 tasks***

Non- Pipelined Processing:

Stages/Clock Cycles	1	2	3	4	5	6	7	8	9	10	11	12
Stage 1	T1				T2				T3			
Stage 2		T1				T2				T3		
Stage 3			T1				T2				T3	
Stage 4				T1				T2				T3

T_1 = nk clock cycles
= 3*4 clock cycles
= 12 clock cycles

When pipelining is not done

Total time to process 'n tasks' in 'non-pipelined processor':

$$T_1 = nk \quad \text{clock cycles}$$

Total time to process k stage pipelined processor

- Let τ_i be the time delay of the circuit in stages S_i and d is the time delay of the latch and τ_m be the maximum stage delay

Then $\tau = \max \{\tau_i\}_1^k + d = \tau_m + d$

Clock skewing, Efficiency and throughput

Pipeline frequency is defined as the:

$$f = 1/\tau$$

If one result come out of the pipeline per cycle, **f represents the maximum throughput** of pipeline, but the actual throughput may be lower than the f depending on the initiation rate of the successive pipelined process which may take more than one cycle.

Clock Skewing: Due to clock skewing the same clock pulse may arrive at different stages with a time offset of s. To avoid this race in two successive stages, we must choose:

$$\tau_m >= \tau_{max} + s \quad \text{and} \quad d <= \tau_{min} - s$$

Where

τ_{max} is time delay of the longest logic path within a stage.

τ_{min} is time delay of the shortest logic path within a stage.

s is the offset and **d** is the latch delay

*in ideal case $\tau_{max} = \tau_m$, $\tau_{min} = d$ and $s = 0$

Speedup factor

The speedup of a k-stage pipeline over an equivalent non-pipelined processor is:

$$S_k\ T_1/T_k = \frac{nk}{k+(n-1)}$$

Efficiency: $S_k/k = n/k+(n-1)$

Pipeline throughput is defined as the no. of tasks performed per unit time and is given by:

$$H_k = \frac{nf}{k+(n-1)}$$

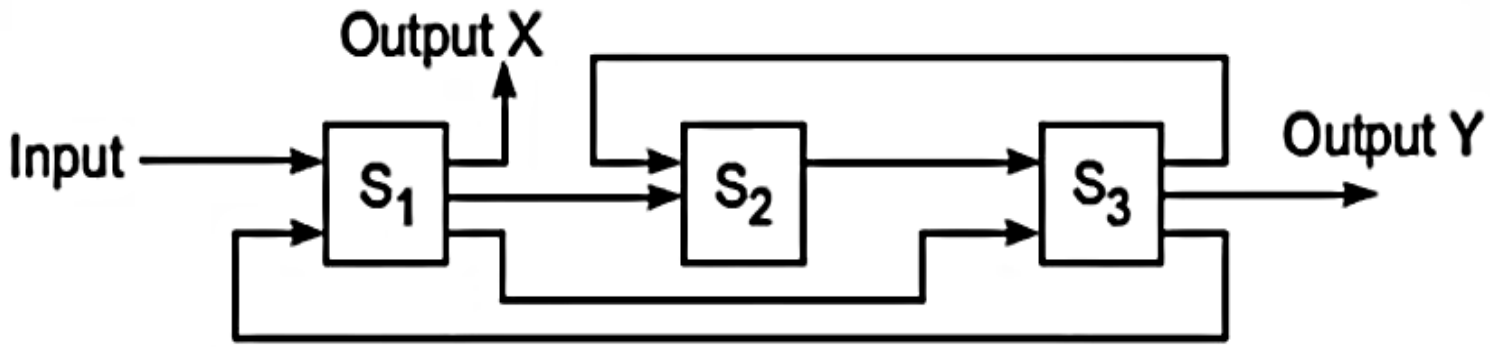
Non-Linear Pipelined Processors

Linear Pipelines are **static pipelines** as they are used to perform ***fixed functions.***

Non-Linear Pipeline are dynamic pipelines. It allows ***feed back*** and ***feed forward connections***, in addition to the ***streamline connections.***

It can be reconfigured to perform ***different (or variable) functions*** at different times i.e. it is ***Multifunctional***

It gives ***more than one output;*** the output of the pipeline is not necessarily from the last stage.



A three-stage non-linear pipeline

Types of connections in non-linear pipeline processors

This pipeline has 3 stages with three types of connections:

1. ***Streamline connections***

   From S1 to S2

   From S2 to S3

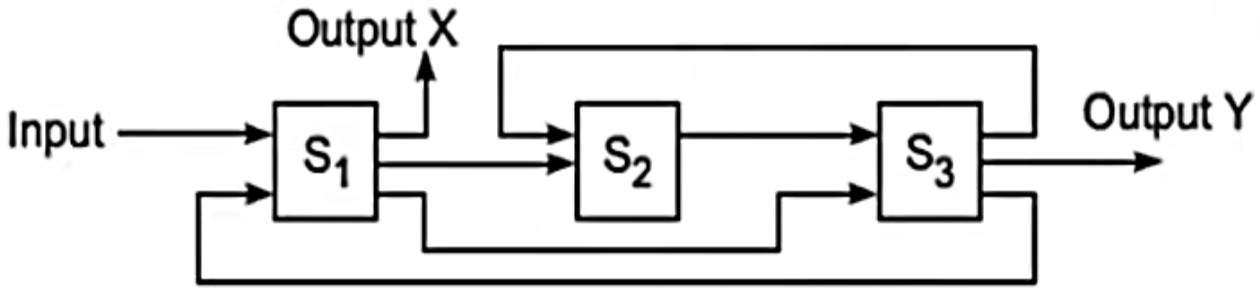
2. ***Feed forward connections***

   From S1 to S3

3. ***Feed back* connections**

   From S3 to S2

   From S3 to S1



A three-stage non-linear pipeline

With these connections, the output of the pipeline is not necessarily from the last stage.

It has two outputs X and Y

Reservation Table

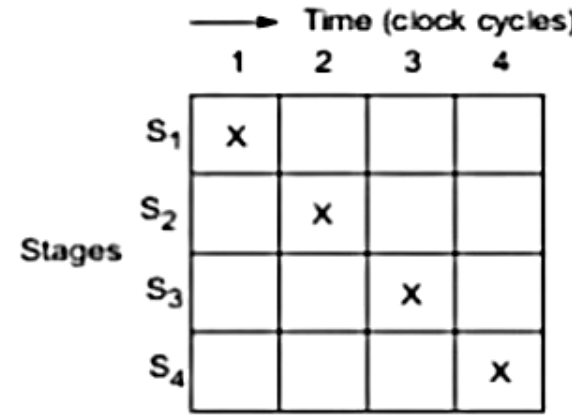
Reservation Table specifies the utilization pattern of successive stages in a pipeline.

Rows represents ***stages*** and **columns** represents ***clock time units***

For Linear Pipeline:

Reservation Table for **Linear Pipeline Processor** follows **diagonal stream line pattern**

Reservation Table for **4-stage linear pipeline**



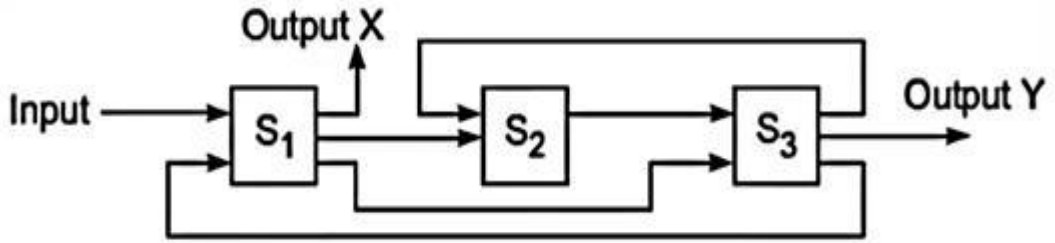
For Non-Linear Pipeline:

Reservation Table for **Non-Linear Pipeline Processor** follows nonlinear pattern.

On the **evaluation of different functions** (like X and Y) **multiple reservation table** can be generated.

Each reservation table displays the flow of data through the pipeline for one complete evaluation of a pipeline.

Reservation table for non-linear pipeline for two functions



Time

Stages	1	2	3	4	5	6	7	8
S_1	X					X		X
S_2		X		X				
S_3			X		X		X	

(b) Reservation table for function X

Time

Stages	1	2	3	4	5	6
S_1	Y				Y	
S_2			Y			
S_3		Y		Y		Y

(c) Reservation table for function Y

Processing Sequence for function 'X'

S1 → S2 → S3 → S2 → S3 → S1 → S3 → S1

Processing Sequence for function 'Y'

S1 → S3 → S2 → S3 → S1 → S3

Multiple check marks in a row means **repeated usage** of the same stage in different cycles

Contiguous check marks in a row means, **extended usage** of a stage over more than one cycle

Multiple check marks in a column means **simultaneous usage** of multiple stages in same cycle

The **total number of clock units** in the reservation table is called **evaluation time** for the given function

Linear vs Non-Linear Pipeline

Linear Pipeline	Non-Linear Pipeline
1. Linear pipeline are static pipeline because they are used to perform fixed functions	1. Non-Linear pipeline are dynamic pipeline because they can be reconfigured to perform variable functions at different times.
2. Linear pipeline allows only streamline connections.	2. Non-Linear pipeline allows feed-forward and feedback connections in addition to the streamline connection.
3. It is relatively easy to partition a given function into a sequence of linearly ordered sub functions.	3. Function partitioning is relatively difficult because the pipeline stages are interconnected with loops in addition to streamline connections.
4. The Output of the pipeline is produced from the last stage.	4. The Output of the pipeline is not necessarily produced from the last stage.
5. The reservation table is trivial in the sense that data flows in linear streamline.	5. The reservation table is non-trivial in the sense that there is no linear streamline for data flows.
6. A Single Reservation table is needed for the evaluation of different functions.	6. Multiple Reservation tables can be generated for the evaluation of different functions.
7. All initiations to a static pipeline use the same reservation table	7. A dynamic pipeline may allow different initiations to follow a mix of reservation tables.

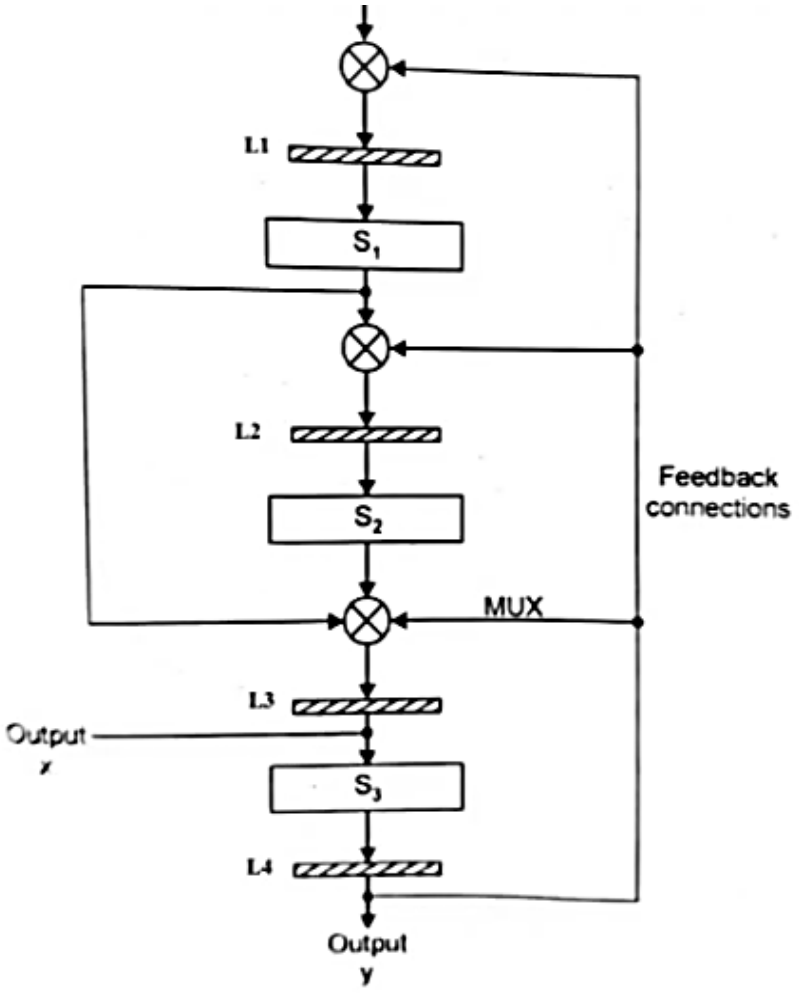
Non-Linear (general)Pipeline Design

In **general pipeline**, the pipeline may have **feed-back connections and feed-forward connections** along with streamline (cascade) connections. So it is **nonlinear.**

*A **general pipeline** may have multiple paths, parallel usage of multiple stages and non-linear flow of data.*

Example: *A General Pipeline with 3-stages*

- It is a **two function pipeline** with function **x** and function **y.**
- There are **three stages** S1, S2, S3.
- The **crossed circles** are **multiplexers (MUX).** They are used for selecting multiple connection paths in evaluating different functions.
- The elements **L1 to L4** are **latches** which helps to just store and forward the input that they receive. These latches are also used as **delays** to synchronize the movement of data from one stage of pipeline to another stage and also they help **avoid** any intermediate **data loss.**



A General Pipeline

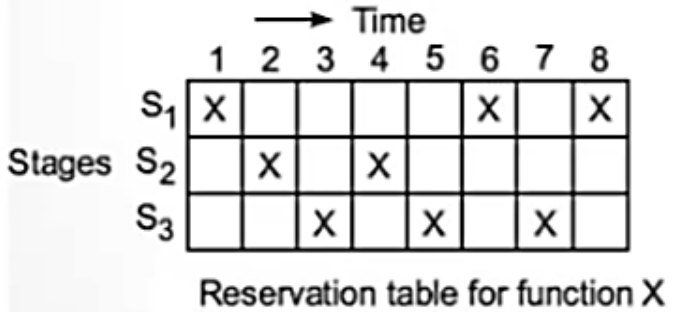
Latency and Collision

Latency:

An **initiation** refers to the **start** of **single function evaluation**

The **number of clock cycles** between **two initiations** of a pipeline is the **latency** between **them.**

Latency values must be **positive integers.**



Reservation table for function X

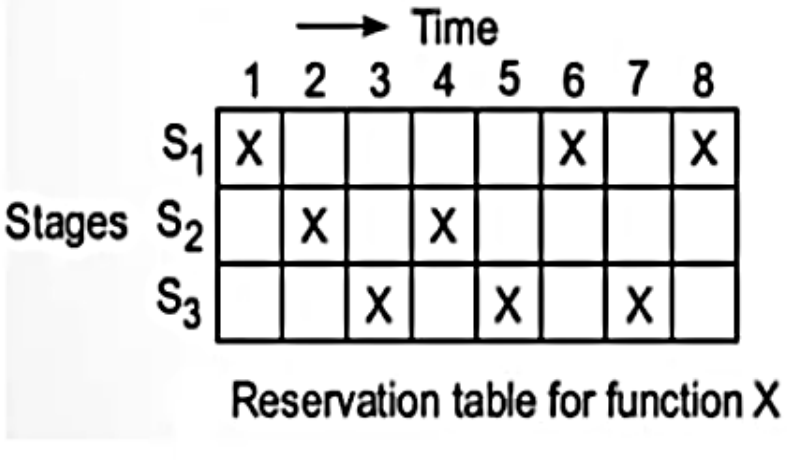
Collision :

An attempt by **two or more initiations** to **use** the **same pipeline stage** at the **same time** is called **collision**

Collision implies **Resource Conflicts** between **two initiations**

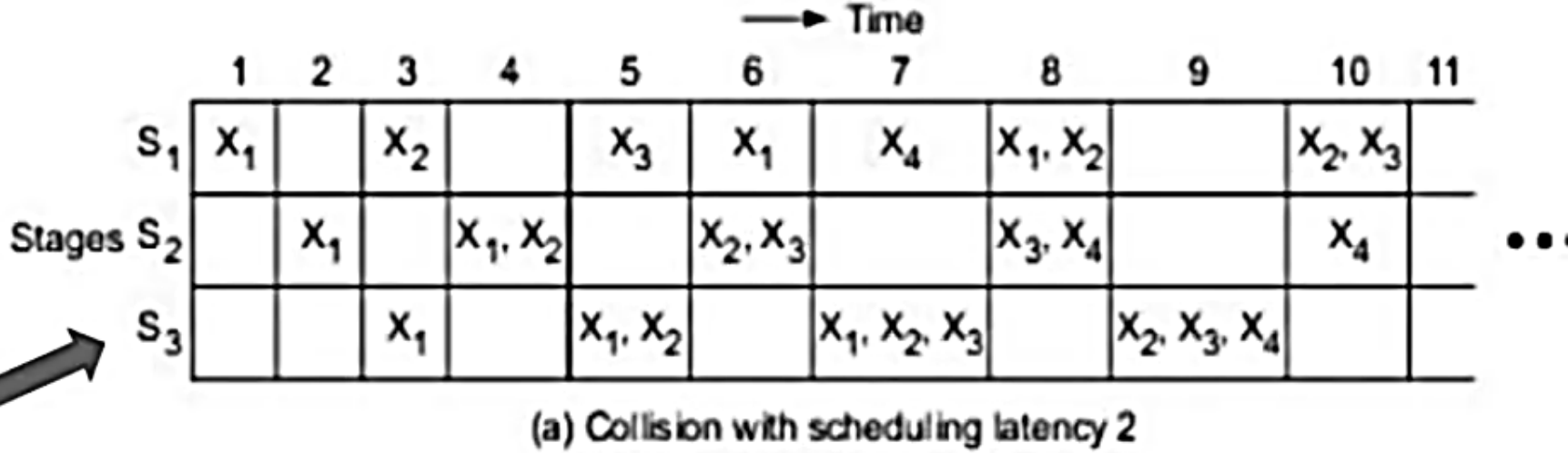
Collisions must be **avoided**

"Some latencies cause collisions and some not."

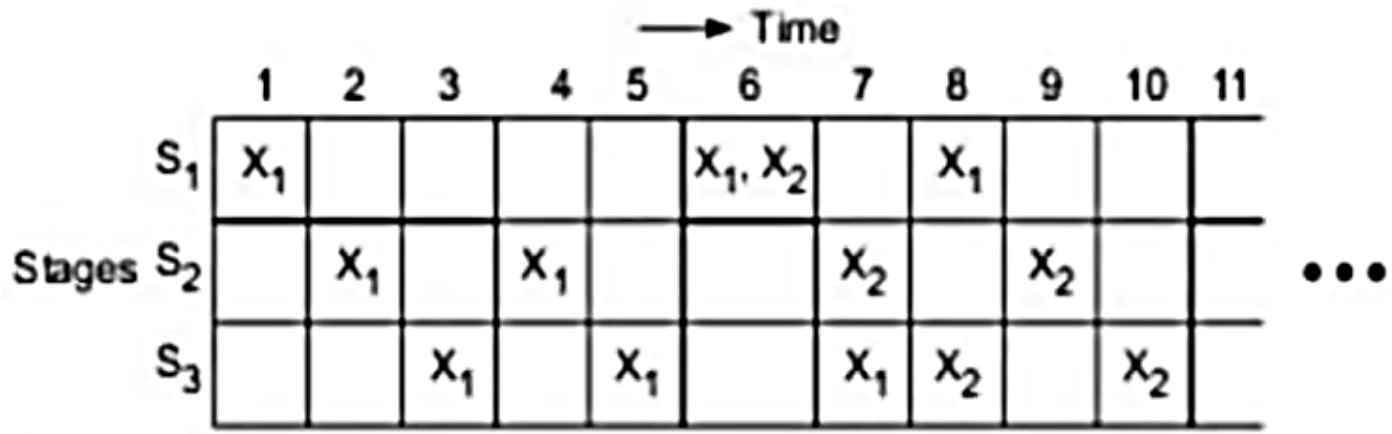


Reservation table for function X

Latencies 2 & 5 cause collision



(a) Collision with scheduling latency 2



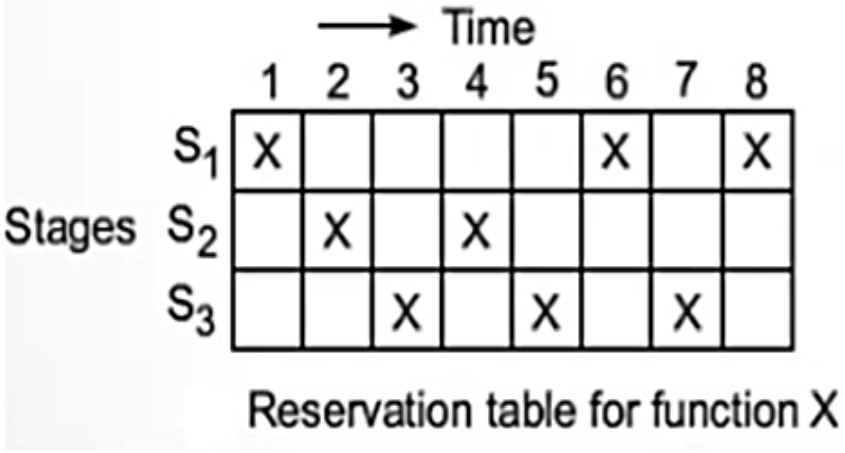
(b) Collision with scheduling latency 5

Forbidden Latencies

Forbidden Latencies:

Latencies that cause collisions are called **forbidden latencies.**

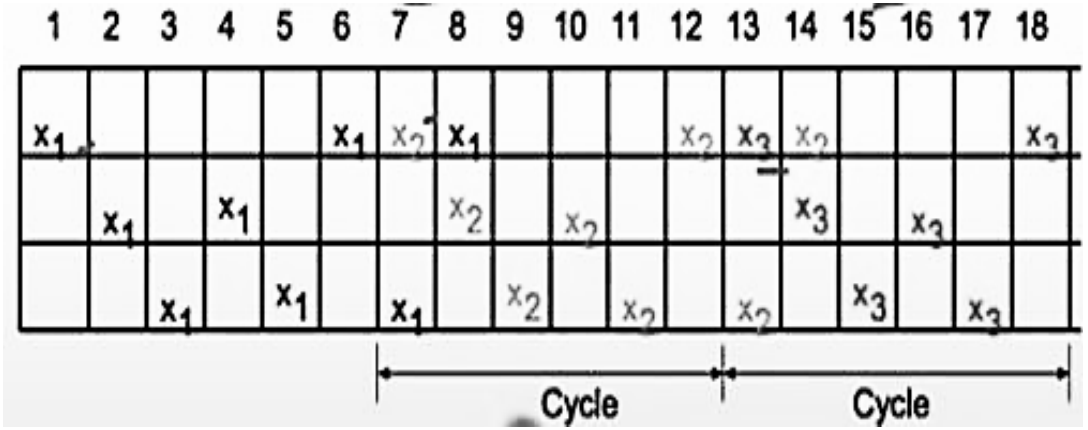
To detect a forbidden latency, one needs simply to check the distance between any 2 checkmarks in the same row of the reservation table.



Consider the reservation table:
S1 has latency {5,2,7}
S2 has latency {2}
S3 has latency {2,4,2}
So, for the given reservation table **forbidden latencies are**
{2, 4, 5, 7}

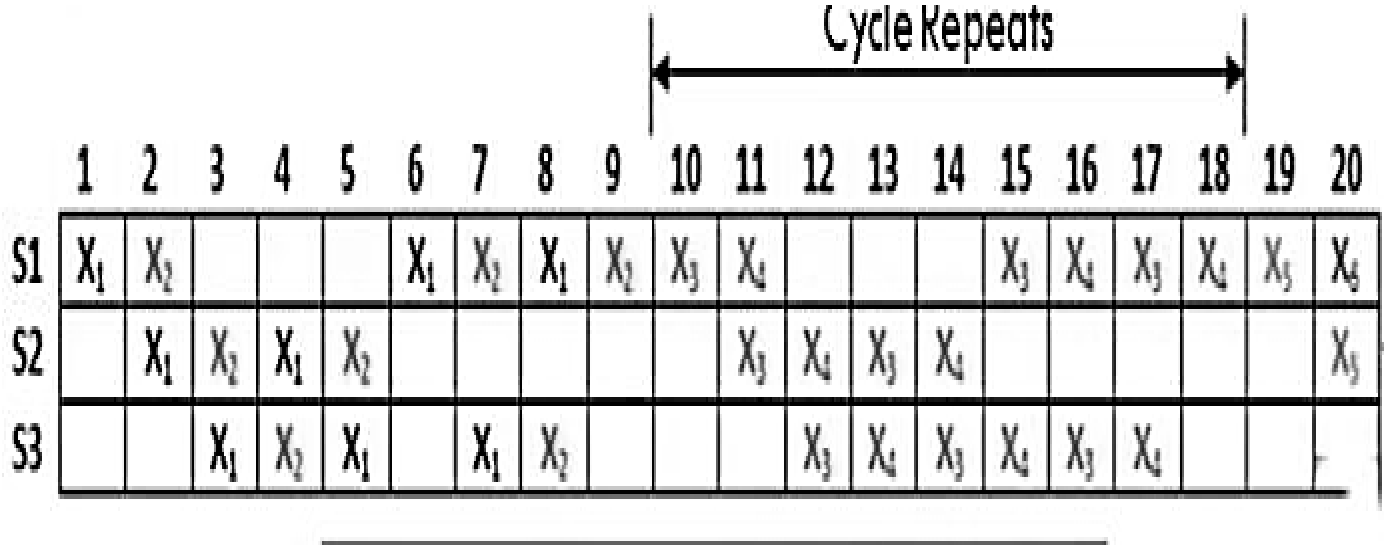
Permissible latencies (also known as latency sequence): that does not cause collision are permissible latencies.
For the given reservation table permissible latencies are,
{1, 3, 6}

Latency cycle: that repeats the same sequence/cycle indefinitely.



With latency 6

With latency 1



Average Latency

- The average latency of a latency cycle is the sum of all latencies divided by number of latencies. (simple average).

For e.g.., the latency cycle (1,8) has average latency of (1+8)/2 = 4.5

Constant cycle: it is a latency cycle that contains only one value of latency (repeated).

Collision free Detection

To avoid collisions and to achieve high throughput, all the tasks awaiting for initiation should be properly scheduled.

When scheduling events in a **nonlinear pipeline**, the **main objective** is:

- **to obtain the *shortest average latency* between initiations without causing collisions**

Following are the steps to obtain this:

1. Collision Vector **(Will be obtained from forbidden latency and permissible latency)**
2. State Diagram **(can be formed with the help of collision vector)**
3. Greedy Cycles **(can be obtained with the help of state diagrams)**
4. MAL (Minimum average Latency)

(lastly, the minimum average latency of greedy cycles will be obtained.)

Collision Vector

Collision vector is an **m-bit binary vector** $C = (C_m C_{m-1} \ldots C_2 C_1)$ that can be obtained from the ***combined set of permissible and forbidden latencies.***

(Where **m<=n-1** in a **n column** reservation table)

C_i=**1** if **latency** i causes a **collision** i.e. for **forbidden latencies**

and C_i=**0** for **permissible latencies.**

Eg: Collision Vector C_X **= (1011010)**

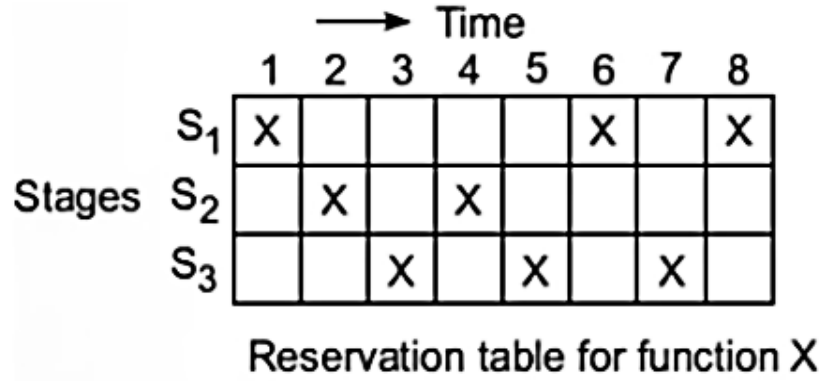
Forbidden latencies = {2, 4, 5, 7}

Maximum forbidden latency = 7, So **m = 7**

So there will be **7 bits** to represent **collision vector**

Collision Vector $C_X = \{C_7\ C_6\ C_5\ C_4\ C_3\ C_2\ C_1\}$

Permissible Latency is the **non-forbidden latency = {1, 3, 6}**



Reservation table for function X

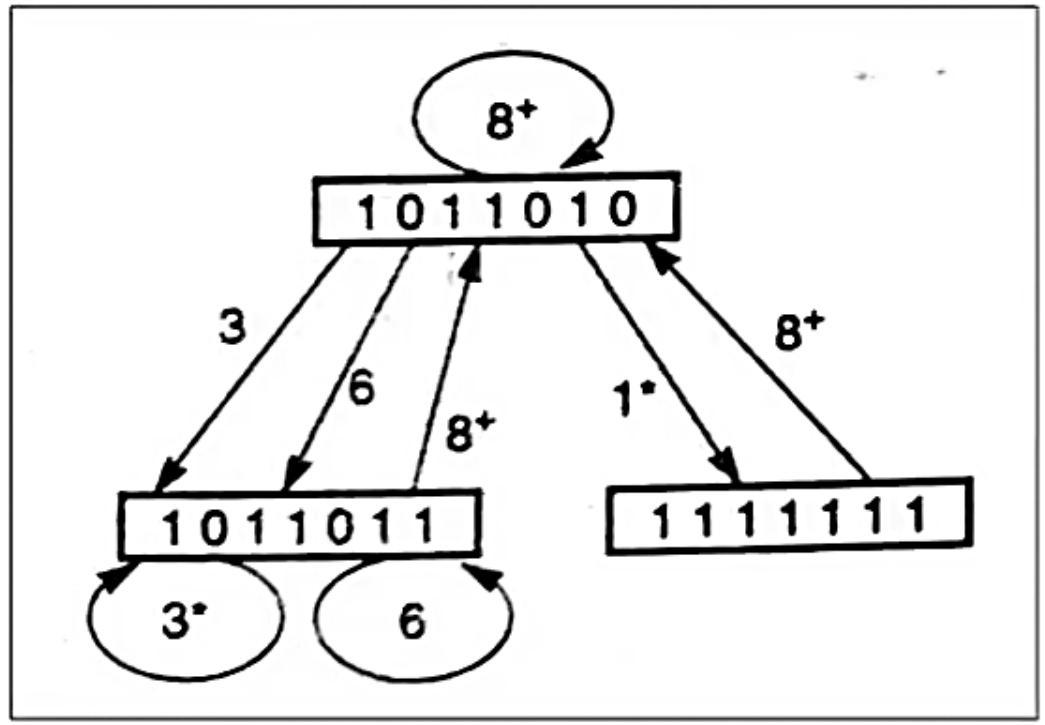
for **Forbidden latencies = {2, 4, 5, 7}**

Collision Vector = $\{C_7\ C_6\ C_5\ C_4\ C_3\ C_2\ C_1\}$	= (1011010)
1 0 1 1 0 1 0	

State Diagram

A **state diagram** is constructed from **Collision Vector**

A **state diagram** will specify the **permissible state transitions** among successive initiations based on the **collision vector.**



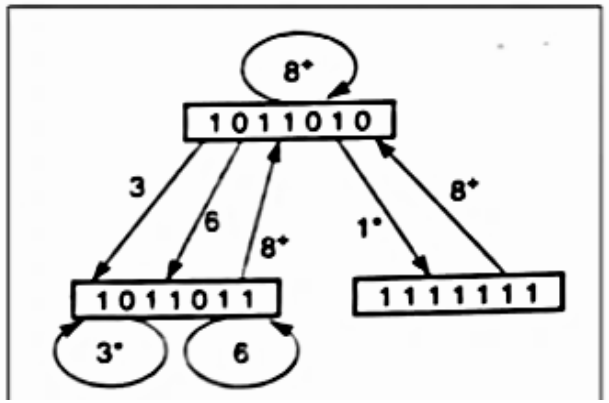
Latency cycles *from the state diagram are (1,8), (1,8,3,8), (3), (6), (8), (3,8), (6,8), (3,6,3).....*

Simple Cycle

- A **Simple Cycle** is a **latency cycle** in which **each state appears only once.**
- In the state diagram only (3), (6), (8), (1, 8), (3, 8), and (6, 8) are simple cycles.
- The cycle (1, 8, 6, 8) is not simple as it travels twice through state (1011010).
- **Some** of the **simple cycles** are **greedy cycles.**

Greedy Cycle

- A **Greedy Cycle** is a **simple cycle** whose edges are all made with **minimum latencies** from their respective starting states.
- The cycle (1, 8) and (3) are greedy cycles.



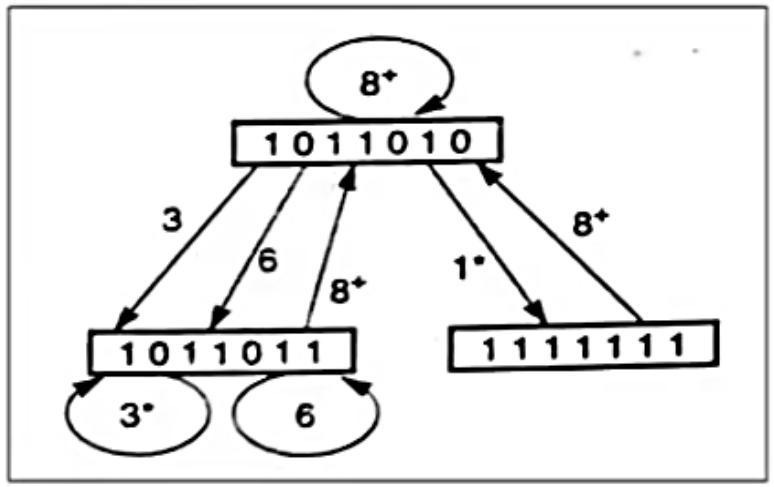
MAL (Minimum Average Latency) is the **minimum average latency** obtained from the greedy cycle.

Eg: ***Avg. latency of greedy cycle (1, 8) = 4.5***

Avg. latency of greedy cycle (3) = 3

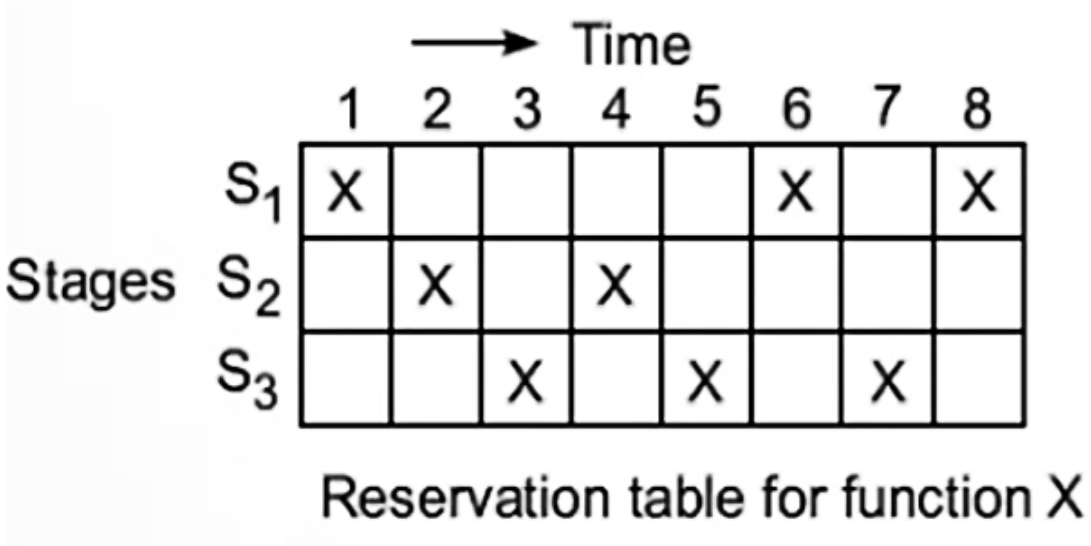
Minimum average latency i.e. MAL= 3

In greedy cycles (1, 8) and (3), the **cycle (3) leads to MAL value 3.**



Note: The minimum-latency edges in the state diagrams are marked with asterisks.

Numerical on collision free detection



For the given reservation table, obtain:

1. What are the forbidden latencies?
2. Draw the state transition diagram.
3. List all the simple cycles and greedy cycles.
4. Determine the minimal average latency (MAL).

Step1: obtain forbidden latencies.

Step2: based on the forbidden latencies find permissible latencies.

Step3: find collision vector based on the highest value of forbidden latency.

For e.g.. If FL maximum value is 7, collision vector C = ($C_7C_6C_5C_4C_3C_2C_1$)

Step4: now draw state diagram based on C (that is initial value of state diagram will be value of collision vector)

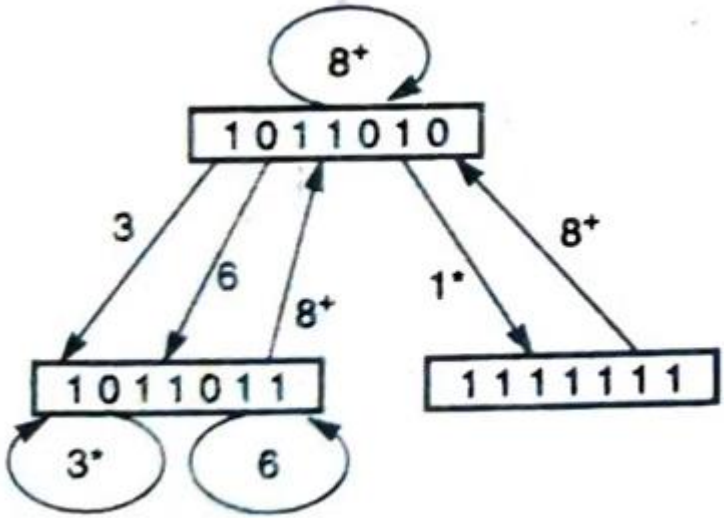
Step5: now find next state of state diagram using the PL values for that…

Step6: Again, find the next state from new states. (in this case ICV will remain same but shifting of new state will be done before bitwise ORing)

To find the collision vector of the next state the collision vector of the present state is shifted right for each transitions (1, 3, 6, 8^+). The shifted collision vector of present state is bitwise-ORed with Initial Collision Vector (ICV).

Note:

1. The **bitwise ORing** of the **shifted version of the present state** with the **initial collision vector** (ICV) is meant *to prevent collisions from future initiations.*
2. When the number of shifts is **m + 1** (7+1 =**8**) **or greater** (**8+**), *all transitions are redirected back to the initial state.*
   - For example, after **eight or more shifts** (denoted as), the next 8+ state must be the initial state, regardless of which state the transition starts from.



Final State Diagram

- **The bitwise Oring of the shifted version of the present state with the initial collision vector (ICV) is meant to prevent collisions**

From future initiations.

- **When the number of shifts is m+1 (7+1) =8 or greater (8+), all transitions are redirected back to the initial state.**

Calculation of throughput

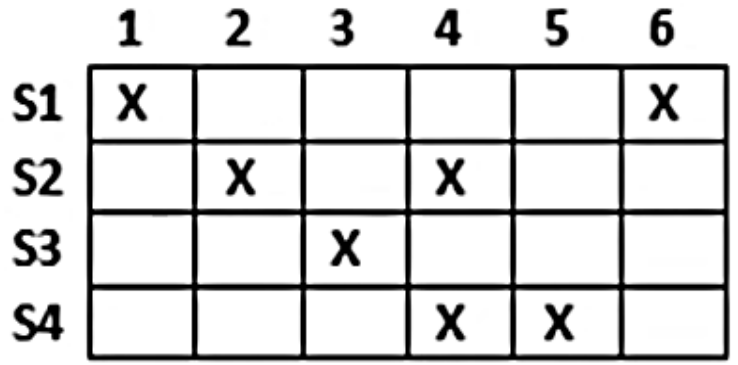
Throughput = $\frac{f}{MAL}$

Where f = 1/τ

τ is the clock pipeline period

Lower bound of MAL = the maximum number of checkmarks in any row of the reservation table.

So **Lower Bound on MAL = 2**

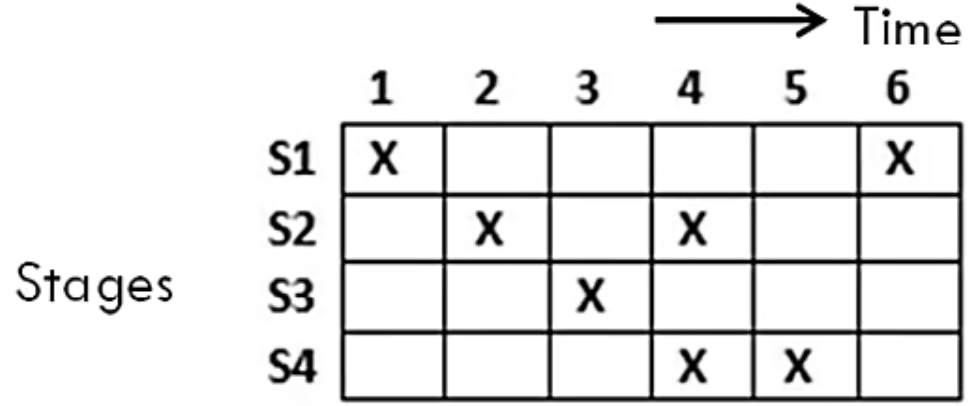


	1	2	3	4	5	6
S1	X					X
S2		X		X		
S3			X			
S4				X	X	

***The value of lower bound is the optimal value of latency so if Lower bound is equal to the average value of latency achieved then only We can say optimal latency is achieved.**

Upper bound of MAL= the number of 1's in the initial collision vector plus 1.

So the **Upper bound on MAL = 3+1= 4**



Suppose ***you are allowed to insert one non-compute delay stage*** *into the given pipeline (for previous question 4)* ***to make latency of 1 permissible*** *in the shortest greedy cycle. The* ***purpose*** *is to yield a* ***new reservation table leading to an optimal latency equal to the lower bound***.

1. *Show the modified reservation table with five rows and seven columns.*
2. *Draw the new state transition diagram for obtaining the optimal cycle.*
3. *List all the simple cycles and greedy cycles from the state diagram.*
4. *Prove that the new MAL equals to the lower bound.*
5. *What is the optimal throughput of this pipeline ?*